Relational Database & SQL Learning Roadmap

A comprehensive, hands-on guide to mastering relational databases, SQL, and practical database programming with Go (no ORMs).

📚 Learning Path

Follow this progression for optimal learning:

1. **Weeks 1-2**: Sections 1-3 (Introduction → Data Retrieval) + exercises/01-sql-basics/ + exercises/02-data-retrieval/
2. **Week 3**: Section 4 (Joins) + exercises/03-joins/
3. **Week 4**: Sections 5-6 (Aggregation & Subqueries) + exercises/04-aggregation/ + exercises/05-subqueries/
4. **Week 5**: Section 7 (Database Design) + Start Airtel project schema design
5. **Week 6**: Section 8 (Advanced SQL) + exercises/06-advanced/
6. **Week 7**: Section 9 (Practical Applications) + Implement Airtel project customer module
7. **Week 8**: Complete Airtel project (billing, reports, analytics modules)
8. **Week 9**: Section 10 (Migrate to PostgreSQL) + Refactor Airtel project

Checkpoint: Before moving to Advanced SQL, ensure you can:

- Write complex multi-table JOINs
- Use aggregate functions with GROUP BY effectively
- Understand when to use subqueries vs JOINs
- Have completed exercises 1-5

---

1. Introduction to Relational Databases

What is a Relational Database?

A **relational database** is a structured collection of data organized into **tables** (also called relations) that can be linked based on common data fields. The relational model was introduced by Edgar F. Codd in 1970 and remains the dominant database paradigm.

Key Characteristics:

- Data organized in tables with rows and columns
- Relationships between tables defined through keys
- Data integrity enforced through constraints
- Queried using SQL (Structured Query Language)

Key Concepts

Tables (Relations)

A table is a collection of related data organized in rows and columns.

```sql
-- Example: A simple users table
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Rows (Records/Tuples)

Each row represents a single record or entity instance.

```sql
-- A single row in the users table
INSERT INTO users (id, username, email)
VALUES (1, 'john_doe', 'john@example.com');
```

Columns (Attributes/Fields)

Each column represents a specific attribute with a defined data type.

Common data types:

- `INTEGER`, `BIGINT`: Whole numbers
- `REAL`, `NUMERIC`, `DECIMAL`: Decimal numbers
- `TEXT`, `VARCHAR(n)`: Strings
- `DATE`, `DATETIME`, `TIMESTAMP`: Date/time values
- `BOOLEAN`: True/false values
- `BLOB`: Binary data

Primary Keys

A **primary key** uniquely identifies each row in a table. It must be:

- **Unique**: No two rows can have the same primary key
- **Not NULL**: Primary key cannot be empty
- **Immutable**: Should not change over time

```sql
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    phone TEXT
);
```

Foreign Keys

A **foreign key** is a column (or set of columns) that references the primary key of another table, establishing relationships between tables.

```sql
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date DATE,
    total_amount DECIMAL(10, 2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

This creates a **one-to-many** relationship: one customer can have many orders.

Benefits of Using Relational Databases

1. **Data Integrity**: Constraints and relationships ensure data accuracy
2. **Reduced Redundancy**: Normalization eliminates duplicate data
3. **ACID Compliance**: Transactions guarantee consistency
4. **Flexibility**: Complex queries can combine data in various ways
5. **Scalability**: Proven technology for handling large datasets
6. **Security**: Fine-grained access control and permissions
7. **Standardization**: SQL is a universal standard across vendors
8. **Data Independence**: Physical storage separated from logical structure

---

2. SQL Basics

What is SQL?

SQL (Structured Query Language) is a standardized programming language for managing and manipulating relational databases. It's declarative—you describe *what* you want, not *how* to get it.

SQL Categories:

- **DDL (Data Definition Language)**: CREATE, ALTER, DROP - define database structure
- **DML (Data Manipulation Language)**: SELECT, INSERT, UPDATE, DELETE - manipulate data
- **DCL (Data Control Language)**: GRANT, REVOKE - manage permissions
- **TCL (Transaction Control Language)**: COMMIT, ROLLBACK, SAVEPOINT - manage transactions

SQL Syntax and Structure

SQL is case-insensitive (keywords can be uppercase or lowercase), but convention uses UPPERCASE for keywords and lowercase for identifiers.

Basic Syntax Rules:

- Statements end with a semicolon (`;`)
- Whitespace is generally ignored (use for readability)
- Comments: `-- single line` or `/* multi-line */`
- String literals use single quotes: `'text'`
- Identifiers (table/column names) can be quoted with double quotes or backticks

Basic SQL Commands

CREATE - Define Database Structure

```sql
-- Create a table
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    category TEXT,
    price DECIMAL(10, 2) NOT NULL CHECK(price >= 0),
    stock_quantity INTEGER DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Create an index for faster queries
CREATE INDEX idx_products_category ON products(category);
```

INSERT - Add Data

```sql
-- Insert a single row
INSERT INTO products (product_name, category, price, stock_quantity)
VALUES ('Laptop', 'Electronics', 899.99, 50);

-- Insert multiple rows
INSERT INTO products (product_name, category, price, stock_quantity) VALUES
    ('Mouse', 'Electronics', 29.99, 200),
    ('Desk Chair', 'Furniture', 199.99, 30),
    ('Notebook', 'Stationery', 4.99, 500);

-- Insert from a SELECT query
INSERT INTO archived_products
SELECT * FROM products WHERE stock_quantity = 0;
```

SELECT - Retrieve Data

```sql
-- Select all columns
SELECT * FROM products;

-- Select specific columns
SELECT product_name, price FROM products;

-- Select with aliases
SELECT
    product_name AS name,
    price AS cost,
    stock_quantity AS "units in stock"
FROM products;

-- Select with expressions
SELECT
    product_name,
    price,
    price * 1.15 AS price_with_tax,
    stock_quantity * price AS inventory_value
FROM products;
```

UPDATE - Modify Data

```sql
-- Update all rows (dangerous! Usually want WHERE clause)
UPDATE products SET price = price * 1.10;

-- Update specific rows
UPDATE products
SET price = 849.99, stock_quantity = 45
WHERE product_name = 'Laptop';

-- Update with expressions
UPDATE products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 1;

-- Update with subquery
UPDATE products
SET price = price * 0.9
WHERE category IN (SELECT category FROM categories WHERE discount_eligible = 1);
```

DELETE - Remove Data

```sql
-- Delete specific rows
DELETE FROM products WHERE stock_quantity = 0;

-- Delete with complex conditions
DELETE FROM products
WHERE category = 'Seasonal' AND created_at < DATE('now', '-1 year');

-- Delete all rows (use with caution!)
DELETE FROM products;
-- Or more efficiently: TRUNCATE TABLE products; (PostgreSQL)
```

⚠️ **Safety Tip**: Always test UPDATE and DELETE with SELECT first:

```sql
-- Test before updating
SELECT * FROM products WHERE stock_quantity = 0;
-- Then update
DELETE FROM products WHERE stock_quantity = 0;
```

Practice Exercise

Create a simple bookstore database:

```sql
-- TODO: Create a books table with:
-- - book_id (primary key)
-- - title
-- - author
-- - isbn
-- - price
-- - publication_year

-- TODO: Insert at least 5 books

-- TODO: Update the price of one book

-- TODO: Delete books published before 2000

-- See exercises/01-sql-basics/ for more practice
```

3. Data Retrieval with SQL

Using SELECT Statements to Query Data

The SELECT statement is your primary tool for retrieving data. Master it thoroughly—it's the foundation of SQL.

Basic SELECT Syntax

```sql
SELECT column1, column2, ...
FROM table_name;
```

SELECT DISTINCT - Remove Duplicates

```sql
-- Get unique categories
SELECT DISTINCT category FROM products;

-- Count unique values
SELECT COUNT(DISTINCT category) AS unique_categories FROM products;

-- Multiple columns (combination must be unique)
SELECT DISTINCT category, price FROM products;
```

Filtering Data with WHERE Clause

The WHERE clause filters rows based on conditions.

Comparison Operators

```sql
-- Equal to
SELECT * FROM products WHERE category = 'Electronics';

-- Not equal to
SELECT * FROM products WHERE category != 'Electronics';
-- Or: WHERE category <> 'Electronics'

-- Greater than, less than
SELECT * FROM products WHERE price > 100;
SELECT * FROM products WHERE stock_quantity <= 10;

-- Between (inclusive)
SELECT * FROM products WHERE price BETWEEN 50 AND 200;
-- Equivalent to: WHERE price >= 50 AND price <= 200
```

Logical Operators

```sql
-- AND - all conditions must be true
SELECT * FROM products
WHERE category = 'Electronics' AND price < 500;

-- OR - at least one condition must be true
SELECT * FROM products
WHERE category = 'Electronics' OR category = 'Furniture';

-- NOT - negates a condition
SELECT * FROM products
WHERE NOT category = 'Electronics';

-- Complex combinations (use parentheses for clarity)
SELECT * FROM products
WHERE (category = 'Electronics' OR category = 'Furniture')
  AND price > 100
  AND stock_quantity > 0;
```

Pattern Matching with LIKE

```sql
-- Wildcards: % (any characters), _ (single character)

-- Starts with 'Lap'
SELECT * FROM products WHERE product_name LIKE 'Lap%';

-- Contains 'book'
SELECT * FROM products WHERE product_name LIKE '%book%';

-- Ends with 'chair'
SELECT * FROM products WHERE product_name LIKE '%chair';

-- Exactly 5 characters
SELECT * FROM products WHERE product_code LIKE '_____';

-- Case-insensitive in SQLite, case-sensitive in PostgreSQL
-- PostgreSQL: use ILIKE for case-insensitive
```

NULL Handling

```sql
-- Find rows where column is NULL
SELECT * FROM products WHERE description IS NULL;

-- Find rows where column is NOT NULL
SELECT * FROM products WHERE description IS NOT NULL;

-- Never use = NULL or != NULL (always false!)
-- WRONG: WHERE description = NULL
-- RIGHT: WHERE description IS NULL
```

IN Operator

```sql
-- Check if value is in a list
SELECT * FROM products
WHERE category IN ('Electronics', 'Furniture', 'Appliances');

-- Equivalent to multiple ORs
SELECT * FROM products
WHERE category = 'Electronics'
   OR category = 'Furniture'
   OR category = 'Appliances';

-- NOT IN
SELECT * FROM products
WHERE category NOT IN ('Clearance', 'Discontinued');

-- IN with subquery
SELECT * FROM products
WHERE category IN (SELECT name FROM categories WHERE active = 1);
```

Sorting Data with ORDER BY

```sql
-- Ascending order (default)
SELECT * FROM products ORDER BY price;
-- Or explicitly: ORDER BY price ASC

-- Descending order
SELECT * FROM products ORDER BY price DESC;

-- Multiple columns
SELECT * FROM products
ORDER BY category ASC, price DESC;

-- Order by expression
SELECT product_name, price, stock_quantity,
       (price * stock_quantity) AS inventory_value
FROM products
ORDER BY inventory_value DESC;

-- Order by column position (not recommended, less readable)
SELECT product_name, price FROM products ORDER BY 2 DESC;

-- NULL handling
SELECT * FROM products ORDER BY description NULLS LAST;
-- SQLite: NULLs come first in ASC, last in DESC
-- PostgreSQL: use NULLS FIRST or NULLS LAST explicitly
```

Limiting Results with LIMIT

```sql
-- Get first 10 rows
SELECT * FROM products ORDER BY price DESC LIMIT 10;

-- Pagination: LIMIT with OFFSET
-- Page 1 (rows 1-10)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 0;

-- Page 2 (rows 11-20)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 10;

-- Page 3 (rows 21-30)
SELECT * FROM products ORDER BY product_id LIMIT 10 OFFSET 20;

-- Formula: OFFSET = (page_number - 1) * page_size

-- Top 5 most expensive products
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;

-- Note: PostgreSQL also supports FETCH FIRST n ROWS ONLY (SQL standard)
```

Complete Example: Complex Query

```sql
-- Find electronics and furniture products
-- priced between $50 and $500
-- that are in stock
-- ordered by price descending
-- show only the top 20 results

SELECT
    product_name,
    category,
    price,
    stock_quantity,
    (price * stock_quantity) AS total_value
FROM products
WHERE category IN ('Electronics', 'Furniture')
  AND price BETWEEN 50 AND 500
  AND stock_quantity > 0
ORDER BY price DESC
LIMIT 20;
```

Practice Exercise

```sql
-- TODO: Find all products containing 'Pro' in the name
-- TODO: Find products with NULL descriptions
-- TODO: List the top 3 least expensive products in each category
-- TODO: Find products where inventory value (price * quantity) > 10000

-- See exercises/02-data-retrieval/ for comprehensive practice
```

---

4. Joining Tables

Understanding Table Relationships

Relational databases derive their power from **relationships** between tables. Three main types:

One-to-Many (1:N)

Most common relationship. One record in Table A relates to many records in Table B.

Example: One customer → many orders

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    email TEXT UNIQUE
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Many-to-Many (M:N)

Each record in Table A can relate to many records in Table B, and vice versa. Requires a **junction table**.

Example: Students ↔ Courses (students take many courses, courses have many students)

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    student_name TEXT NOT NULL
);

CREATE TABLE courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL
);

-- Junction table
CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE,
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id),
    UNIQUE(student_id, course_id)
);
```

One-to-One (1:1)

Each record in Table A relates to exactly one record in Table B. Less common; often used to split tables for performance or security.

Example: User → User Profile Details

```sql
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,
    username TEXT NOT NULL
);

CREATE TABLE user_profiles (
    user_id INTEGER PRIMARY KEY,
    bio TEXT,
    avatar_url TEXT,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

Types of Joins

Sample tables for examples:

```sql
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY,
    dept_name TEXT NOT NULL
);

CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT NOT NULL,
    dept_id INTEGER,
    salary DECIMAL(10, 2),
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

-- Sample data
INSERT INTO departments (dept_id, dept_name) VALUES
    (1, 'Engineering'),
    (2, 'Sales'),
    (3, 'Marketing');

INSERT INTO employees (emp_id, emp_name, dept_id, salary) VALUES
    (101, 'Alice', 1, 80000),
    (102, 'Bob', 1, 75000),
    (103, 'Charlie', 2, 60000),
    (104, 'Diana', NULL, 50000);  -- No department assigned
```

INNER JOIN

Returns only rows where there's a match in both tables. Most common join type.

```sql
-- List employees with their department names
SELECT
    employees.emp_name,
    employees.salary,
    departments.dept_name
FROM employees
INNER JOIN departments ON employees.dept_id = departments.dept_id;

-- Result: Alice, Bob, Charlie (Diana excluded - no dept)
--         Marketing dept excluded (no employees)
```

Shorter syntax with aliases:

```sql
SELECT
    e.emp_name,
    e.salary,
    d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Multiple join conditions:

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id
    AND d.dept_name != 'Sales';
```

LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table, and matching rows from the right table. If no match, right side columns are NULL.

```
-- List all employees, including those without a department
SELECT
    e.emp_name,
    e.salary,
    d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- Result: All employees appear
--         Diana's dept_name will be NULL
```

Use case: Find employees without a department assignment

```
SELECT e.emp_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;

-- Result: Diana
```

RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table, and matching rows from the left table. Mirror of LEFT JOIN.

```
-- List all departments, including those with no employees
SELECT
    d.dept_name,
    e.emp_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

-- Result: All departments appear
--         Marketing will have NULL emp_name

-- Note: SQLite doesn't support RIGHT JOIN
-- Rewrite as LEFT JOIN by swapping table order:
SELECT d.dept_name, e.emp_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

FULL OUTER JOIN

Returns all rows from both tables. Where there’s no match, the missing side contains NULL.

```sql
-- List all employees and all departments
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;

-- Result: Alice, Bob, Charlie, Diana (no dept), Marketing (no employees)

-- Note: SQLite doesn't support FULL OUTER JOIN
-- Emulate with UNION of LEFT JOIN and RIGHT JOIN:
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

CROSS JOIN (Cartesian Product)

Returns every possible combination of rows from both tables. Use cautiously—can produce enormous result sets!

```sql
-- Every employee paired with every department
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;

-- Result: 4 employees × 3 departments = 12 rows
```

Use case: Generate combinations (e.g., size × color for product variants)

SELF JOIN

A table joined with itself. Useful for hierarchical data.

```sql
-- Employees table with manager reference
CREATE TABLE employees_v2 (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT NOT NULL,
    manager_id INTEGER,
    FOREIGN KEY (manager_id) REFERENCES employees_v2(emp_id)
);

INSERT INTO employees_v2 VALUES
    (1, 'CEO', NULL),
    (2, 'VP Engineering', 1),
    (3, 'VP Sales', 1),
    (4, 'Engineer', 2),
    (5, 'Engineer', 2);

-- List employees with their manager's name
SELECT
    e.emp_name AS employee,
    m.emp_name AS manager
FROM employees_v2 e
LEFT JOIN employees_v2 m ON e.manager_id = m.emp_id;
```

Using JOINs to Combine Data from Multiple Tables

Three-Table Join

```sql
-- Orders database
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_name TEXT,
    quantity INTEGER,
    price DECIMAL(10, 2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

-- Find all order details for each customer
SELECT
    c.customer_name,
    o.order_id,
    o.order_date,
    oi.product_name,
    oi.quantity,
    oi.price,
    (oi.quantity * oi.price) AS line_total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id
ORDER BY c.customer_name, o.order_date;
```

Join with Aggregation

```sql
-- Total order value per customer
SELECT
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS total_orders,
    COALESCE(SUM(oi.quantity * oi.price), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC;
```

Join with WHERE vs Join Condition

```sql
-- Filter in JOIN condition (for outer joins)
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
    AND o.order_date >= '2024-01-01';  -- Condition in JOIN

-- vs filter in WHERE clause
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01'  -- Condition in WHERE
   OR o.order_date IS NULL;

-- These are DIFFERENT!
-- First: All customers, with orders only from 2024
-- Second: Customers who have orders from 2024
```

Practice Exercise

```sql
-- TODO: Create a library database (books, authors, borrowers, loans)
-- TODO: Write joins to find:
--   1. All books with their authors
--   2. Borrowers who haven't returned books (overdue)
--   3. Authors with the most borrowed books
--   4. Books never borrowed

-- See exercises/03-joins/ for comprehensive practice
```

---

5. Aggregating Data

Aggregate Functions

Aggregate functions perform calculations on multiple rows and return a single value.

COUNT - Count Rows

```sql
-- Count all rows
SELECT COUNT(*) FROM products;

-- Count non-NULL values in a column
SELECT COUNT(description) FROM products;

-- Count distinct values
SELECT COUNT(DISTINCT category) FROM products;

-- Count with condition
SELECT COUNT(*) FROM products WHERE price > 100;
```

SUM - Add Up Values

```sql
-- Total inventory value
SELECT SUM(price * stock_quantity) AS total_inventory_value
FROM products;

-- Sum with filter
SELECT SUM(price) FROM products WHERE category = 'Electronics';

-- Note: SUM ignores NULL values
```

AVG - Calculate Average

```sql
-- Average product price
SELECT AVG(price) AS average_price FROM products;

-- Average by category
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category;

-- Note: AVG ignores NULL values
-- To include NULLs as zeros: AVG(COALESCE(price, 0))
```

MIN and MAX - Find Extremes

```sql
-- Cheapest and most expensive products
SELECT
    MIN(price) AS cheapest,
    MAX(price) AS most_expensive
FROM products;

-- Works on dates, strings too
SELECT
    MIN(created_at) AS first_product,
    MAX(created_at) AS latest_product
FROM products;
```

Combining Multiple Aggregates

```sql
-- Comprehensive statistics
SELECT
    COUNT(*) AS total_products,
    COUNT(DISTINCT category) AS total_categories,
    SUM(stock_quantity) AS total_units,
    AVG(price) AS average_price,
    MIN(price) AS min_price,
    MAX(price) AS max_price,
    SUM(price * stock_quantity) AS total_value
FROM products;
```

Grouping Data with GROUP BY

GROUP BY divides rows into groups based on column values, then applies aggregate functions to each group.

Basic GROUP BY

```sql
-- Count products per category
SELECT
    category,
    COUNT(*) AS product_count
FROM products
GROUP BY category;

-- Multiple aggregates per group
SELECT
    category,
    COUNT(*) AS product_count,
    AVG(price) AS avg_price,
    SUM(stock_quantity) AS total_stock
FROM products
GROUP BY category
ORDER BY product_count DESC;
```

GROUP BY Multiple Columns

```sql
-- Assuming products have a supplier_id
SELECT
    category,
    supplier_id,
    COUNT(*) AS product_count,
    AVG(price) AS avg_price
FROM products
GROUP BY category, supplier_id
ORDER BY category, supplier_id;
```

GROUP BY with WHERE

WHERE filters rows **before** grouping.

```sql
-- Average price by category, only for in-stock products
SELECT
    category,
    AVG(price) AS avg_price
FROM products
WHERE stock_quantity > 0  -- Filter before grouping
GROUP BY category;
```

Important Grouping Rules

Rule: Every column in SELECT (except aggregate functions) must appear in GROUP BY.

```sql
-- WRONG: product_name not in GROUP BY
SELECT category, product_name, COUNT(*)
FROM products
GROUP BY category;

-- CORRECT: Only aggregates and grouped columns
SELECT category, COUNT(*) AS count
FROM products
GROUP BY category;
```

Filtering Groups with HAVING

HAVING filters groups **after** aggregation. Use it to filter on aggregate function results.

```sql
-- Categories with more than 10 products
SELECT
    category,
    COUNT(*) AS product_count
FROM products
GROUP BY category
HAVING COUNT(*) > 10;

-- Categories with average price above $100
SELECT
    category,
    AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 100
ORDER BY avg_price DESC;
```

HAVING vs WHERE

```sql
-- WHERE: filters individual rows before grouping
-- HAVING: filters groups after aggregation

-- Find categories with > 5 expensive products (price > 100)
SELECT
    category,
    COUNT(*) AS expensive_product_count
FROM products
WHERE price > 100              -- Filter rows first
GROUP BY category
HAVING COUNT(*) > 5;            -- Then filter groups

-- Order of execution:
-- 1. FROM: Get products table
-- 2. WHERE: Keep only products with price > 100
-- 3. GROUP BY: Group by category
-- 4. HAVING: Keep only groups with count > 5
-- 5. SELECT: Calculate aggregates
-- 6. ORDER BY: Sort results (if specified)
```

Complex Aggregation Example

```sql
-- Sales analysis by month and category
SELECT
    strftime('%Y-%m', order_date) AS month,
    category,
    COUNT(DISTINCT order_id) AS order_count,
    SUM(quantity) AS units_sold,
    SUM(quantity * unit_price) AS revenue,
    AVG(unit_price) AS avg_sale_price
FROM order_items oi
INNER JOIN products p ON oi.product_id = p.product_id
INNER JOIN orders o ON oi.order_id = o.order_id
WHERE order_date >= DATE('now', '-1 year')
GROUP BY strftime('%Y-%m', order_date), category
HAVING revenue > 1000
ORDER BY month DESC, revenue DESC;
```

Useful Aggregate Patterns

```sql
-- Percentage of total
SELECT
    category,
    COUNT(*) AS count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM products), 2) AS percentage
FROM products
GROUP BY category;

-- Running totals (requires window functions or self-join)
-- See Advanced SQL section

-- NULL handling in aggregates
SELECT
    category,
    COUNT(*) AS total_products,
    COUNT(description) AS products_with_description,
    COUNT(*) - COUNT(description) AS products_without_description
FROM products
GROUP BY category;
```

Practice Exercise

```sql
-- TODO: Find the total sales by customer and year
-- TODO: Find customers who spent more than $1000
-- TODO: Find the average order value per month
-- TODO: Identify categories with declining sales (compare quarters)

-- See exercises/04-aggregation/ for comprehensive practice
```

---

6. Subqueries and Nested Queries

What are Subqueries?

A **subquery** (or nested query) is a query embedded within another query. The subquery executes first, and its result is used by the outer query.

Benefits:

- Break complex problems into smaller steps
- Improve readability
- Perform calculations not possible with simple queries

Basic syntax:

```sql
SELECT column1
FROM table1
WHERE column2 = (SELECT column3 FROM table2 WHERE condition);
```

Using Subqueries in SELECT, FROM, and WHERE Clauses

Subqueries in WHERE Clause

Most common usage. Compare values against subquery results.

```sql
-- Find products more expensive than average
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Find customers who placed orders
SELECT customer_name
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);

-- The opposite: customers who never ordered
SELECT customer_name
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders WHERE customer_id IS NOT NULL);
-- Note: Be careful with NULL in NOT IN!
```

Subqueries with Comparison Operators

```sql
-- Products in the most expensive category (by average price)
SELECT product_name, category, price
FROM products
WHERE category = (
    SELECT category
    FROM products
    GROUP BY category
    ORDER BY AVG(price) DESC
    LIMIT 1
);

-- ALL: Compare to all values
-- Products more expensive than ALL products in 'Stationery'
SELECT product_name, price
FROM products
WHERE price > ALL (SELECT price FROM products WHERE category = 'Stationery');

-- Equivalent to:
WHERE price > (SELECT MAX(price) FROM products WHERE category = 'Stationery');

-- ANY (or SOME): Compare to any value
-- Products more expensive than ANY product in 'Stationery'
SELECT product_name, price
FROM products
WHERE price > ANY (SELECT price FROM products WHERE category = 'Stationery');

-- Equivalent to:
WHERE price > (SELECT MIN(price) FROM products WHERE category = 'Stationery');
```

Subqueries in SELECT Clause

Returns a single value for each row in the outer query.

```sql
-- Show each product with percentage of category average
SELECT
    product_name,
    category,
    price,
    (SELECT AVG(price) FROM products p2 WHERE p2.category = p1.category) AS category_avg,
    price - (SELECT AVG(price) FROM products p2 WHERE p2.category = p1.category) AS price_diff
FROM products p1;

-- Count of orders per customer
SELECT
    customer_name,
    (SELECT COUNT(*) FROM orders WHERE customer_id = c.customer_id) AS order_count
FROM customers c;
```

Subqueries in FROM Clause (Derived Tables)

Treat a subquery result as a temporary table.

```sql
-- Average sales by month, then find months above overall average
SELECT month, monthly_revenue
FROM (
    SELECT
        strftime('%Y-%m', order_date) AS month,
        SUM(total_amount) AS monthly_revenue
    FROM orders
    GROUP BY strftime('%Y-%m', order_date)
) AS monthly_sales
WHERE monthly_revenue > (
    SELECT AVG(monthly_revenue)
    FROM (
        SELECT SUM(total_amount) AS monthly_revenue
        FROM orders
        GROUP BY strftime('%Y-%m', order_date)
    )
);

-- With aliasing for readability
SELECT ms.month, ms.revenue
FROM (
    SELECT
        strftime('%Y-%m', order_date) AS month,
        SUM(total_amount) AS revenue
    FROM orders
    GROUP BY month
) AS ms
WHERE ms.revenue > 10000;
```

EXISTS and NOT EXISTS

Test whether a subquery returns any rows.

```sql
-- Customers who have placed at least one order
SELECT customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

-- Categories that have products in stock
SELECT DISTINCT category
FROM products p1
WHERE EXISTS (
    SELECT 1 FROM products p2
    WHERE p2.category = p1.category AND p2.stock_quantity > 0
);

-- Customers who never ordered
SELECT customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

-- EXISTS is often more efficient than IN for large datasets
```

Correlated vs. Non-Correlated Subqueries

Non-Correlated Subqueries

Independent of outer query. Executes once.

```sql
-- Find products priced above average (non-correlated)
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Execution:
-- 1. Calculate AVG(price) once
-- 2. Filter products with that value
```

Correlated Subqueries

References columns from outer query. Executes once per outer row (potentially slower).

```sql
-- Products priced above their category average (correlated)
SELECT product_name, category, price
FROM products p1
WHERE price > (
    SELECT AVG(price)
    FROM products p2
    WHERE p2.category = p1.category  -- References outer query!
);

-- Execution:
-- 1. For each product in p1:
-- 2.   Calculate AVG(price) for that product's category
-- 3.   Compare product price to that average

-- Find each customer's most recent order
SELECT c.customer_name, o.order_date, o.total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date = (
    SELECT MAX(order_date)
    FROM orders o2
    WHERE o2.customer_id = c.customer_id
);
```

Performance Considerations

```sql
-- Correlated subquery (potentially slow)
SELECT p.product_name, p.price,
    (SELECT AVG(price) FROM products WHERE category = p.category) AS cat_avg
FROM products p;

-- Often better: Use JOIN or window functions
SELECT p.product_name, p.price, avg_prices.cat_avg
FROM products p
INNER JOIN (
    SELECT category, AVG(price) AS cat_avg
    FROM products
    GROUP BY category
) avg_prices ON p.category = avg_prices.category;
```

Complex Example: Top N Per Group

```sql
-- Top 3 most expensive products per category

-- Using correlated subquery
SELECT product_name, category, price
FROM products p1
WHERE (
    SELECT COUNT(*)
    FROM products p2
    WHERE p2.category = p1.category AND p2.price > p1.price
) < 3
ORDER BY category, price DESC;

-- Modern approach: Window functions (see Advanced SQL)
SELECT *
FROM (
    SELECT
        product_name,
        category,
        price,
        ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC) AS rank
    FROM products
)
WHERE rank <= 3;
```

Practice Exercise

```sql
-- TODO: Find employees earning more than their department average
-- TODO: List departments with above-average employee count
-- TODO: Find products that have never been ordered
-- TODO: Get each customer's order history with running order count

-- See exercises/05-subqueries/ for comprehensive practice
```

---

7. Database Design and Normalization

Principles of Database Design

Good database design ensures:

- **Data integrity**: Accuracy and consistency
- **Efficiency**: Fast queries and minimal storage
- **Maintainability**: Easy to update and extend
- **Scalability**: Handles growth gracefully

Key Design Principles

1. **Identify entities**: Things you need to store (customers, products, orders)
2. **Define relationships**: How entities relate (customer *places* order)
3. **Choose primary keys**: Unique identifier for each entity
4. **Normalize**: Eliminate redundancy
5. **Add constraints**: Enforce business rules
6. **Consider performance**: Index frequently queried columns

Entity-Relationship (ER) Modeling

Before creating tables, model your domain:

```
Customer (1) ----< has many >---- (N) Order
Order (1) ----< contains >---- (N) OrderItem
OrderItem (N) ----< references >---- (1) Product
```

Normalization: 1NF, 2NF, 3NF

Normalization is the process of organizing data to reduce redundancy and improve integrity.

First Normal Form (1NF)

Rules:

1. Each column contains atomic (indivisible) values
2. No repeating groups
3. Each row is unique (has a primary key)

Violations:

```sql
-- BAD: Multiple values in one column
CREATE TABLE orders_bad (
    order_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    products TEXT  -- "Laptop, Mouse, Keyboard" - NOT atomic!
);

-- BAD: Repeating groups
CREATE TABLE orders_bad2 (
    order_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    product1 TEXT,
    product2 TEXT,
    product3 TEXT  -- What if customer orders 4 products?
);
```

Fixed (1NF):

```sql
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_name TEXT
);

CREATE TABLE order_items (
    item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_name TEXT,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

Second Normal Form (2NF)

Rules:

1. Must be in 1NF
2. No partial dependencies (all non-key columns fully depend on the entire primary key)

Applies to composite primary keys.

Violation:

```sql
-- Composite key: (order_id, product_id)
CREATE TABLE order_items_bad (
    order_id INTEGER,
    product_id INTEGER,
    product_name TEXT,      -- Depends only on product_id, not order_id!
    product_price DECIMAL,  -- Depends only on product_id!
    quantity INTEGER,       -- Depends on both (good)
    PRIMARY KEY (order_id, product_id)
);
```

Fixed (2NF):

```sql
-- Separate products table
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT,
    product_price DECIMAL
);

CREATE TABLE order_items (
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Third Normal Form (3NF)

Rules:

1. Must be in 2NF
2. No transitive dependencies (non-key columns don't depend on other non-key columns)

Violation:

```sql
CREATE TABLE employees_bad (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT,
    dept_id INTEGER,
    dept_name TEXT,     -- Depends on dept_id, not emp_id directly!
    dept_location TEXT  -- Transitive dependency!
);
```

Fixed (3NF):

```sql
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY,
    dept_name TEXT,
    dept_location TEXT
);

CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    emp_name TEXT,
    dept_id INTEGER,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Benefits of Normalization

- **Eliminates redundancy**: Store each fact once
- **Prevents anomalies**:
  - **Update**: Change data in one place, not many
  - **Insert**: Can't add incomplete data
  - **Delete**: Removing data doesn't lose other information
- **Saves storage**: No duplicate data
- **Maintains integrity**: Constraints ensure consistency

Example: Normalizing a Flat Table

Starting point (unnormalized):

```sql
CREATE TABLE sales_flat (
    invoice_id INTEGER,
    invoice_date DATE,
    customer_name TEXT,
    customer_email TEXT,
    customer_phone TEXT,
    product_name TEXT,
    product_category TEXT,
    product_price DECIMAL,
    quantity INTEGER
);

-- Problems:
-- 1. Duplicate customer info for each product
-- 2. Duplicate product info for each sale
-- 3. Can't add customer without an order
-- 4. Deleting last order removes customer data
```

Normalized (3NF):

```sql
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    customer_email TEXT UNIQUE,
    customer_phone TEXT
);

CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    product_category TEXT,
    product_price DECIMAL NOT NULL
);

CREATE TABLE invoices (
    invoice_id INTEGER PRIMARY KEY,
    invoice_date DATE NOT NULL,
    customer_id INTEGER NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE invoice_items (
    item_id INTEGER PRIMARY KEY,
    invoice_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    unit_price DECIMAL NOT NULL,  -- Historical price at time of sale
    FOREIGN KEY (invoice_id) REFERENCES invoices(invoice_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Denormalization and When to Use It

Denormalization intentionally introduces redundancy to improve query performance.

When to denormalize:

- Read-heavy workloads (queries far outnumber updates)
- Complex joins causing slow queries
- Aggregated data accessed frequently
- Data warehousing / analytics

Techniques:

```sql
-- Add computed/redundant columns
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date DATE,
    -- Denormalized: Store customer name (avoid join)
    customer_name TEXT,
    -- Denormalized: Store total (avoid computing)
    order_total DECIMAL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Summary tables
CREATE TABLE daily_sales_summary (
    sale_date DATE PRIMARY KEY,
    total_orders INTEGER,
    total_revenue DECIMAL,
    avg_order_value DECIMAL
);
-- Updated nightly by a batch job

-- Materialized views (PostgreSQL)
CREATE MATERIALIZED VIEW product_sales_stats AS
SELECT
    product_id,
    COUNT(*) AS times_sold,
    SUM(quantity) AS units_sold,
    SUM(quantity * unit_price) AS total_revenue
FROM order_items
GROUP BY product_id;

-- Refresh periodically
REFRESH MATERIALIZED VIEW product_sales_stats;
```

Trade-offs:

- ✅ Faster reads
- ❌ Slower writes (update multiple places)
- ❌ More storage
- ❌ Risk of inconsistency
- ❌ More complex update logic

Best practice: Normalize first, denormalize only when measurements show it's necessary.

---

8. Advanced SQL Concepts

Indexing for Performance Optimization

Index: A data structure that improves query speed by allowing fast lookups. Like a book index—find pages without reading the whole book.

How Indexes Work

- Create a sorted copy of column(s) with pointers to rows
- Queries use index to find rows quickly (logarithmic time vs. linear scan)
- Trade-off: Faster reads, slower writes (must update index)

Creating Indexes

```sql
-- Single-column index
CREATE INDEX idx_products_category ON products(category);

-- Now this is fast:
SELECT * FROM products WHERE category = 'Electronics';

-- Multi-column index (order matters!)
CREATE INDEX idx_employees_dept_salary ON employees(dept_id, salary);

-- Helps queries filtering by dept_id, or both dept_id and salary
-- Does NOT help queries filtering only by salary

-- Unique index (enforces uniqueness)
CREATE UNIQUE INDEX idx_users_email ON users(email);
-- Similar to UNIQUE constraint

-- Partial index (SQLite 3.8+, PostgreSQL)
CREATE INDEX idx_products_low_stock ON products(product_id)
WHERE stock_quantity < 10;
-- Only indexes low-stock products, smaller and faster
```

When to Index

Good candidates:

- Columns in WHERE clauses
- Columns in JOIN conditions
- Columns in ORDER BY / GROUP BY
- Foreign keys
- Columns with high selectivity (many unique values)

Avoid indexing:

- Small tables (full scan is fast enough)
- Columns rarely queried
- Columns with low selectivity (e.g., boolean, small enums)
- Tables with heavy write workload

Checking Index Usage

```sql
-- SQLite: See query plan
EXPLAIN QUERY PLAN
SELECT * FROM products WHERE category = 'Electronics';

-- Look for "USING INDEX" vs "SCAN TABLE"

-- PostgreSQL: EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT * FROM products WHERE category = 'Electronics';
```

Index Maintenance

```sql
-- Drop unused index
DROP INDEX idx_products_category;

-- Rebuild index (SQLite: automatic, PostgreSQL: REINDEX)
REINDEX INDEX idx_products_category;

-- List all indexes (SQLite)
SELECT name, tbl_name FROM sqlite_master WHERE type = 'index';

-- List indexes on a table (PostgreSQL)
\d tablename
```

Transactions and ACID Properties

Transaction: A sequence of operations treated as a single unit. Either all succeed (commit) or all fail (rollback).

ACID Properties

1. **Atomicity**: All-or-nothing. No partial execution.
2. **Consistency**: Database moves from one valid state to another.
3. **Isolation**: Concurrent transactions don't interfere.
4. **Durability**: Committed data survives crashes.

Using Transactions

```sql
-- SQLite/PostgreSQL: Start transaction
BEGIN TRANSACTION;

-- Or simply: BEGIN;

-- Perform operations
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

-- Check if operations succeeded
-- If yes: Commit (make permanent)
COMMIT;

-- If no: Rollback (undo all changes)
ROLLBACK;
```

Example: Bank Transfer

```sql
-- Without transaction (DANGEROUS!)
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- What if crash happens here? Money disappears!
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

-- With transaction (SAFE)
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

-- Check balances haven't gone negative
SELECT balance FROM accounts WHERE account_id = 1 AND balance < 0;
-- If found, rollback; otherwise commit

COMMIT;
```

Isolation Levels

Control how transactions see each other's changes.

```sql
-- PostgreSQL: Set isolation level
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Levels (most to least isolated):

1. **SERIALIZABLE**: Full isolation, slowest
2. **REPEATABLE READ**: Prevents non-repeatable reads
3. **READ COMMITTED**: Prevents dirty reads (default in PostgreSQL)
4. **READ UNCOMMITTED**: Minimal isolation, fastest (not in PostgreSQL)

Savepoints

Create checkpoints within a transaction.

```sql
BEGIN TRANSACTION;

INSERT INTO orders (customer_id, order_date) VALUES (1, '2024-01-15');

SAVEPOINT before_items;

INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 101, 2);
INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 102, 1);

-- Oops, mistake in items!
ROLLBACK TO SAVEPOINT before_items;

-- Re-insert corrected items
INSERT INTO order_items (order_id, product_id, quantity) VALUES (1, 103, 5);

COMMIT;  -- Commits orders and corrected items
```

Stored Procedures and Triggers

Note: SQLite has limited support. PostgreSQL has full support.

Triggers

Automatically execute code when events occur (INSERT, UPDATE, DELETE).

PostgreSQL Example:

```sql
-- Audit table
CREATE TABLE audit_log (
    log_id SERIAL PRIMARY KEY,
    table_name TEXT,
    action TEXT,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    user_name TEXT
);

-- Trigger function
CREATE OR REPLACE FUNCTION log_product_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log (table_name, action, user_name)
        VALUES ('products', 'INSERT', current_user);
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log (table_name, action, user_name)
        VALUES ('products', 'UPDATE', current_user);
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log (table_name, action, user_name)
        VALUES ('products', 'DELETE', current_user);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Create trigger
CREATE TRIGGER products_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON products
FOR EACH ROW
EXECUTE FUNCTION log_product_changes();
```

SQLite Example (limited):

```sql
-- Update timestamp automatically
CREATE TRIGGER update_product_timestamp
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
    UPDATE products SET updated_at = CURRENT_TIMESTAMP
    WHERE product_id = NEW.product_id;
END;
```

Stored Procedures (PostgreSQL)

Reusable SQL code blocks.

```sql
-- Create a stored procedure
CREATE OR REPLACE FUNCTION get_customer_orders(c_id INTEGER)
RETURNS TABLE(order_id INTEGER, order_date DATE, total DECIMAL) AS $$
BEGIN
    RETURN QUERY
    SELECT o.order_id, o.order_date, o.total_amount
    FROM orders o
    WHERE o.customer_id = c_id
    ORDER BY o.order_date DESC;
END;
$$ LANGUAGE plpgsql;

-- Call it
SELECT * FROM get_customer_orders(123);

-- Procedure with side effects
CREATE OR REPLACE PROCEDURE apply_discount(
    p_category TEXT,
    p_discount DECIMAL
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE products
    SET price = price * (1 - p_discount)
    WHERE category = p_category;

    COMMIT;
END;
$$;

-- Call it
CALL apply_discount('Clearance', 0.20);
```

Views (All databases)

Virtual tables based on queries.

```sql
-- Create view
CREATE VIEW high_value_customers AS
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING total_spent > 1000;

-- Use like a table
SELECT * FROM high_value_customers ORDER BY total_spent DESC;

-- Drop view
DROP VIEW high_value_customers;
```

---

9. Practical Applications

Setting up SQLite

Installation

macOS: Pre-installed. Verify:

```
sqlite3 --version
```

Update to latest (optional):

```
brew install sqlite3
```

Basic Usage

```
# Create/open database
sqlite3 mydata.db

# In SQLite shell:
.help                    # Show commands
.tables                  # List tables
.schema table_name       # Show table structure
.mode column             # Pretty output
.headers on              # Show column names
.quit                    # Exit

# Run SQL file
sqlite3 mydata.db < schema.sql

# Dump database
sqlite3 mydata.db .dump > backup.sql
```

SQLite in Go

```go
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/mattn/go-sqlite3"
)

func main() {
    db, err := sql.Open("sqlite3", "./mydata.db")
    if err != nil {
        panic(err)
    }
    defer db.Close()

    // Create table
    _, err = db.Exec(`
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT UNIQUE
        )
    `)
    if err != nil {
        panic(err)
    }

    // Insert
    result, err := db.Exec("INSERT INTO users (name, email) VALUES (?, ?)",
        "Alice", "alice@example.com")
    if err != nil {
        panic(err)
    }

    id, _ := result.LastInsertId()
    fmt.Printf("Inserted user with ID: %d\n", id)

    // Query
    rows, err := db.Query("SELECT id, name, email FROM users")
    if err != nil {
        panic(err)
    }
    defer rows.Close()

    for rows.Next() {
        var id int
        var name, email string
        rows.Scan(&id, &name, &email)
        fmt.Printf("%d: %s <%s>\n", id, name, email)
    }
}
```

Setting up PostgreSQL

Installation

macOS:

```bash
brew install postgresql@16
brew services start postgresql@16

# Create database
createdb mydb
```

Basic Usage

```
# Connect
psql mydb

# In psql:
\?                      # Help
\l                      # List databases
\dt                     # List tables
\d table_name           # Describe table
\q                      # Quit

# Run SQL file
psql mydb < schema.sql

# Dump database
pg_dump mydb > backup.sql
```

PostgreSQL in Go

```go
package main

import (
    "database/sql"
    "fmt"
    _ "github.com/lib/pq"
)

func main() {
    connStr := "user=username dbname=mydb sslmode=disable"
    db, err := sql.Open("postgres", connStr)
    if err != nil {
        panic(err)
    }
    defer db.Close()

    // Use $1, $2 placeholders (not ?)
    _, err = db.Exec(`
        CREATE TABLE IF NOT EXISTS users (
            id SERIAL PRIMARY KEY,
            name TEXT NOT NULL,
            email TEXT UNIQUE
        )
    `)

    // Insert with RETURNING
    var id int
    err = db.QueryRow(`
        INSERT INTO users (name, email)
        VALUES ($1, $2)
        RETURNING id
    `, "Bob", "bob@example.com").Scan(&id)

    fmt.Printf("Inserted user with ID: %d\n", id)
}
```

Best Practices for Database Management and Security

Connection Management

```go
// Configure connection pool
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(5)
db.SetConnMaxLifetime(5 * time.Minute)
```

Prevent SQL Injection

```
// NEVER do this (vulnerable to SQL injection):
query := fmt.Sprintf("SELECT * FROM users WHERE email = '%s'", userInput)
db.Query(query)

// ALWAYS use parameterized queries:
db.Query("SELECT * FROM users WHERE email = ?", userInput)  // SQLite
db.Query("SELECT * FROM users WHERE email = $1", userInput) // PostgreSQL
```

Error Handling

```
// Check for specific errors
import "github.com/lib/pq"

_, err := db.Exec("INSERT INTO users (email) VALUES ($1)", email)
if err != nil {
    if pqErr, ok := err.(*pq.Error); ok {
        if pqErr.Code == "23505" { // Unique violation
            return fmt.Errorf("email already exists")
        }
    }
    return err
}
```

Transactions in Go

```
tx, err := db.Begin()
if err != nil {
    return err
}
defer tx.Rollback() // Rollback if not committed

_, err = tx.Exec("UPDATE accounts SET balance = balance - ? WHERE id = ?", 100, 1)
if err != nil {
    return err
}

_, err = tx.Exec("UPDATE accounts SET balance = balance + ? WHERE id = ?", 100, 2)
if err != nil {
    return err
}

return tx.Commit()
```

Database Migrations

Version control your schema changes.

```
-- migrations/001_create_users.up.sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

-- migrations/001_create_users.down.sql
DROP TABLE users;

-- migrations/002_add_users_phone.up.sql
ALTER TABLE users ADD COLUMN phone TEXT;

-- migrations/002_add_users_phone.down.sql
ALTER TABLE users DROP COLUMN phone;
```

Use tools like `golang-migrate`:

```
migrate -database "postgres://localhost/mydb?sslmode=disable" -path ./migrations up
```

Security Checklist

- ✅ Use parameterized queries (prevent SQL injection)
- ✅ Principle of least privilege (minimal database permissions)
- ✅ Encrypt connections (TLS/SSL)
- ✅ Encrypt sensitive data at rest
- ✅ Regular backups
- ✅ Audit logging
- ✅ Keep database software updated
- ✅ Strong password policies
- ✅ Network isolation (firewall rules)
- ✅ Input validation (before it reaches database)

---

10. SQLite → PostgreSQL Migration Guide

Key Differences

Feature	SQLite	PostgreSQL
Data types	Flexible, dynamic	Strict, static
Auto-increment	`AUTOINCREMENT`	`SERIAL` or `IDENTITY`
Booleans	0/1 integers	TRUE/FALSE
Date/Time	TEXT/INTEGER	DATE, TIMESTAMP types
String concat	`||`	`||` or `CONCAT()`
LIMIT	`LIMIT n OFFSET m`	`LIMIT n OFFSET m` or `FETCH`
RIGHT JOIN	Not supported	Supported
FULL OUTER JOIN	Not supported	Supported
Placeholders	`?`	`$1, $2, $3`
RETURNING	Limited	Full support

Schema Migration Example

SQLite:

```sql
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    is_active INTEGER DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

PostgreSQL:

```sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Query Adaptations

```sql
-- Date functions
-- SQLite:
SELECT date('now')
SELECT strftime('%Y-%m', created_at)

-- PostgreSQL:
SELECT CURRENT_DATE
SELECT TO_CHAR(created_at, 'YYYY-MM')

-- String manipulation
-- SQLite:
SELECT SUBSTR(name, 1, 5)

-- PostgreSQL:
SELECT SUBSTRING(name, 1, 5)

-- Conditional logic (works in both):
SELECT
    CASE
        WHEN price < 100 THEN 'cheap'
        WHEN price < 500 THEN 'moderate'
        ELSE 'expensive'
    END AS price_category
FROM products;
```

Go Code Changes

```go
// SQLite
import _ "github.com/mattn/go-sqlite3"
db, _ := sql.Open("sqlite3", "./mydata.db")
db.Exec("INSERT INTO users (name) VALUES (?)", name)

// PostgreSQL
import _ "github.com/lib/pq"
db, _ := sql.Open("postgres", "user=user dbname=mydb sslmode=disable")
db.Exec("INSERT INTO users (name) VALUES ($1)", name)

// Write database-agnostic code:
type DB interface {
    Exec(query string, args ...interface{}) (sql.Result, error)
    Query(query string, args ...interface{}) (*sql.Rows, error)
}

// Use abstraction layer to handle different placeholders
```

🎯 Next Steps

1. **Complete exercises** in order: exercises/
2. **Start Airtel project**: projects/airtel-network/
3. **Read SETUP.md**: SETUP.md for environment configuration
4. **Practice daily**: Consistency is key to mastery

Recommended Resources

- **SQLite Documentation**: https://www.sqlite.org/docs.html
- **PostgreSQL Tutorial**: https://www.postgresqltutorial.com/
- **SQL Practice**: https://sqlzoo.net/, https://leetcode.com/problemset/database/
- **Go database/sql**: https://go.dev/doc/database/
- **Visualization tool**: DBeaver, pgAdmin, or TablePlus

Learning Tips

- **Write SQL by hand**: Don't rely on ORMs or generators initially
- **Analyze query plans**: Understand performance implications
- **Use real datasets**: Kaggle, data.gov
- **Build projects**: Nothing beats hands-on experience
- **Review others' code**: GitHub, Stack Overflow
- **Benchmark queries**: Measure before optimizing

Good luck on your database learning journey! 🚀

Questions or found an issue? See the exercises and project READMEs for more guidance.